

AIVM: Proof of AI

The Settlement Primitive for Verifiable Machine Computation on Lux

Zach Kelling*

Hanzo Industries Lux Industries Zoo Labs Foundation
research@lux.network

June 2026

Abstract

We present the AIVM (AI Virtual Machine) and its settlement primitive, *Proof of AI* (PoAI): a cryptographic proof that an AI computation *graph* ran correctly. Inference, embedding, and training are all computation graphs — training is simply a forward pass composed with its backward pass — so PoAI is *one* proof system, not three. The proof rests on Freivalds’ matrix-product verifier evaluated over the prime field $\mathbb{F}_p$, $p = 2^{61} - 1$, on the *exact-integer* accumulator of the engine’s quantized `int8` matmul path. Because integer arithmetic is bit-exact across CPUs, GPUs, and backends, the verifier has zero false-reject; its soundness error is information-theoretic, $1/p \approx 2^{-61}$ per challenge vector, giving $\leq 2^{-122}$ at two challenges and inheriting no computational hardness assumption — hence post-quantum by construction. A prover commits each proof-bearing matrix multiplication as a KECCAK-Merkle leaf over its exact-integer operands; a beacon-selected leaf is opened and Freivalds-checked. Whole-graph soundness then follows by induction over a typed, well-formed trace. The same commitment is reproduced byte-for-byte by the Rust prover, the Go verifier, and a Solidity on-chain verifier; we exhibit a unit test in which a proof emitted by the Go path verifies inside the EVM unchanged. The AIVM is the Lux network’s settlement layer for this work: it reconciles minting across every Lux chain through a global, chain-independent compute-claim ledger, sources cross-chain state through a trustless post-quantum validator-quorum relay verified by an ML-DSA precompile, bonds and automatically slashes compute providers, and is governed by a leaderless, sortition-selected quorum of node-operator language models. The native coin is fair-mined under a Bitcoin-shaped schedule (a one-billion hard cap, a four-year halving, and a deflationary fee-burn tail), but the work that mints it is *useful, cryptographically verified AI computation* rather than wasted hashing. Every property in this paper is grounded in shipped, tested code: the soundness core passes 22 Go tests and 27 Rust tests; the on-chain settlement and governance layer passes 256 Solidity tests across 20 suites, all green.

Contents

1	Introduction	2
1.1	Contributions	3
1.2	The shape of the system	3
1.3	Threat model	4
2	The Soundness Core: Freivalds over a Prime Field	4
2.1	Why a matrix product is the right primitive	4
2.2	The field and the exact-integer accumulator	4
2.3	Soundness	5

*zach@lux.network

2.4	Challenge unpredictability	6
2.5	What the core proves, and what it does not	6
2.6	Verified test evidence for the core	6
3	The Transcript: Committing a Forward Pass	6
3.1	Commitment	7
3.2	Challenge and opening	7
3.3	Sound forward pass: the returned value is the committed value	7
4	Whole-Graph Composition: One Proof for Inference, Embedding, and Training	8
4.1	The gap the graph closes	8
4.2	The two checks	8
4.3	Global soundness	8
4.4	Training is a graph, not a separate proof system	9
4.5	Adversarial test evidence	9
5	Tri-Language Byte Parity	10
5.1	One hash, one field, one serialization	10
5.2	The on-chain witness	10
5.3	Proven parity	10
6	The AIVM as the Lux Settlement Layer	11
6.1	Mining a proof into a coin	11
6.2	The global no-double-mint ledger	12
6.3	Trustless, permissionless, post-quantum cross-chain state	12
6.4	Bonded providers and automated slashing	12
7	Governance: A Leaderless Quorum of Operator LLMs	13
7.1	Sortition: leaderless committee selection	13
7.2	Categorical decisions: AIGovernor	13
7.3	Continuous values: AIParams	14
7.4	From a settled decision to an arbitrary on-chain action	14
7.5	Governance as a second proof of useful cognition	14
8	Economics: Bitcoin-Shaped, Fair-Mined on Useful Work	15
8.1	The issuance schedule	15
8.2	Deflation	15
8.3	Why this is the next Bitcoin	15
9	Verified Test Evidence (Aggregate)	16
10	Post-Quantum Primitives	16
11	Related Work	17
12	Conclusion	17

1 Introduction

A blockchain settles claims about value because anyone can cheaply verify them. Bitcoin settles “this work was done” by making the work (a partial hash preimage) trivial to check and expensive to produce; the work itself is otherwise useless. The open question for a decentralized AI network

is whether one can settle a far more valuable claim — “this AI computation was performed correctly” — with the same asymmetry: cheap to verify, expensive to fake, and *the work is the product*.

The AIVM answers yes. Its settlement primitive, Proof of AI, makes a single observation precise: a modern neural network forward pass is roughly 95% matrix multiplications $C = A \cdot B$, and a matrix product is exactly the object Freivalds’ 1977 algorithm verifies probabilistically an order of magnitude cheaper than recomputation [1]. If the proof is anchored on the *exact integer* accumulator of a quantized model — not a floating-point shadow that disagrees across hardware — the check becomes bit-exact, deterministic, and information-theoretically sound. From this one primitive the entire economic machine follows: a unit of verified computation mints a fair-launch subsidy exactly once, network-wide; a fabricated computation is caught and the prover’s bond is slashed; and the proof, the commitment, and the field arithmetic are identical in the prover (Rust), the off-chain watcher (Go), and the on-chain verifier (Solidity).

1.1 Contributions

1. **One proof for inference, embedding, and training (§4).** We define a typed computation trace whose well-formedness plus per-operation local correctness imply, by induction, that the committed output equals the deterministic forward pass on the committed input. Training is the same machinery: a gradient step is a longer graph over the same operations (Theorem 5). Data-sharing is orthogonal — the graph’s committed input *is* its data provenance.
2. **An information-theoretic, post-quantum soundness core (§2).** Freivalds over $\mathbb{F}_p$, $p = 2^{61} - 1$, on an exact-integer accumulator, with $\leq 2^{-122}$ soundness at two challenges and zero false-reject (Theorems 1, 3). The argument cites no computational assumption, so a quantum adversary gains nothing.
3. **Tri-language byte parity (§5).** A prover-side Rust commitment, a Go verifier, and a Solidity on-chain verifier that share one hash, one field, and one canonical serialization, proven by golden cross-language tests and by an EVM test that verifies a Go-produced opening unchanged.
4. **The AIVM as Lux settlement (§6–§7).** A global no-double-mint compute ledger, a trustless permissionless post-quantum cross-chain root relay, capacity-proportional bonding with automated fraud-slashing, and a leaderless sortition-quorum governance layer that settles canonical decisions and executes arbitrary approved operations behind a timelock.
5. **Bitcoin-shaped, fair-mined economics on useful work (§8).** A capped, halving, deflationary issuance schedule mined entirely by verified cognition.

1.2 The shape of the system

The canonical contract implementation lives once, at `luxfi/standard` under `contracts/ai/` (split by concern into `governance/`, `mining/`, `token/`, `compute/`, `relay/`, `registry/`; license BSD-3-Clause), and is consumed everywhere via the `@luxfi/standard/ai/` import path (e.g. `@luxfi/standard/ai/governance/AIGovernor.sol`); there is no second implementation. The soundness core is mirrored three ways: the prover in `hanzo-engine` (`src/poi.rs`, `poi_forward.rs`, `poi_graph.rs`, `poi_transcript.rs`), the verifier in `luxfi/crypto` (`crypto/poi/`), and the on-chain verifier in Solidity (`ComputeWitnessLib.sol`). The remainder of this paper walks the system bottom-up: the field arithmetic and Freivalds core; the per-matmul transcript; the whole-graph composition; the cross-language parity; and finally the settlement, governance, post-quantum, and economic layers that turn a proof into money.

1.3 Threat model

We assume compute providers may be Byzantine: they may return fabricated outputs, claim outputs they never computed, splice unrelated activations into a trace, replay one piece of work to many chains, claim hardware they do not possess, or collude to extract the subsidy without performing the work. We assume the underlying KECCAK-256 hash is collision- and preimage-resistant and that the standardized post-quantum signature ML-DSA (FIPS 204) is unforgeable. We do *not* assume any trusted hardware enclave, any trusted relayer, any privileged sequencer, or any honest-majority computational hardness for the *soundness* of a single proof: that soundness is information-theoretic. We do assume an honest super-majority of the registered governance operator population for the *liveness and policy* layer, and an unpredictable post-commitment public beacon (a future block hash or VRF output) for challenge derivation and committee sortition.

2 The Soundness Core: Freivalds over a Prime Field

2.1 Why a matrix product is the right primitive

Let a proof-bearing layer compute $C = A \cdot B$ with $A \in \mathbb{Z}^{t \times k}$, $B \in \mathbb{Z}^{k \times n}$, $C \in \mathbb{Z}^{t \times n}$. Recomputing C to check it costs $O(t \cdot k \cdot n)$ — exactly the work the prover did, so verification would be no cheaper than redoing the inference. Freivalds’ insight is to check the product against a random vector $r \in \mathbb{F}_p^n$:

$$A \cdot (B \cdot r) \stackrel{?}{=} C \cdot r, \quad (1)$$

which costs $O(tk + kn + tn)$ — three matrix-vector products, an order of magnitude cheaper than the matrix-matrix product. This asymmetry is the whole point: the model is enormous, the witness is one opened matmul, and the verifier does a fraction of the prover’s work.

2.2 The field and the exact-integer accumulator

We evaluate (1) over the Mersenne prime field $\mathbb{F}_p$ with

$$p = 2^{61} - 1. \quad (2)$$

The choice is deliberate. Each operand is an `int8` weight or activation reduced into $[0, p)$; the field is 61 bits, so a single 128-bit multiply of two reduced elements never overflows and a modular reduction by a Mersenne prime is cheap. In the on-chain mirror, p is small enough that Solidity’s `mulmod/addmod` stay exact under it.

Crucially, the operands are taken from the *exact integer* accumulator of the engine’s quantized matmul — an $i8 \times i8 \rightarrow i64$ whole- K accumulation with no cross-block floating-point rescale. The Rust prover documents and implements this as the proof-bearing GEMM (`poi.rs`, `exact_matmul`):

Listing 1: The exact-integer matmul; identical in Go (`crypto/poi.ExactMatmul`).

```
pub fn exact_matmul(a: &Mat, b: &Mat) -> Mat {
    let (t, k, n) = (a.rows, a.cols, b.cols);
    let mut c = vec![0i64; t * n];
    for i in 0..t { for j in 0..n {
        let mut acc: i64 = 0;
        for x in 0..k { acc += a.data[i*k + x] * b.data[x*n + j]; }
        c[i*n + j] = acc;
    }}
    Mat::new(t, n, c)
}
```

Property 1 (Cross-backend determinism). *Integer matrix multiplication is associative and commutative over $\mathbb{Z}$ and incurs no rounding. Therefore two honest machines — regardless of CPU, GPU, SIMD width, or reduction order — produce the bit-identical accumulator C , hence the bit-identical operands the verifier checks.*

This is the property the entire scheme stands on. A floating-point matmul would produce platform-dependent low bits, and the verifier would have to tolerate a fuzz window that an adversary could hide a fabrication inside. By committing the integer accumulator and dequantizing it only as a final public affine map (§3.3), the proof checks the *same* value the layer returns.

2.3 Soundness

Theorem 1 (Freivalds soundness over $\mathbb{F}_p$). *Fix A, B, C over $\mathbb{F}_p$ with $C \neq A \cdot B$. For r drawn uniformly from $\mathbb{F}_p^n$,*

$$\Pr_r[A(Br) = Cr] \leq \frac{1}{p}.$$

Proof. Let $D = C - A \cdot B \neq 0$ over $\mathbb{F}_p$. Then $A(Br) = Cr \iff Dr = 0$. Pick an entry $D_{ij} \neq 0$. Row i of Dr is the linear form $L(r) = \sum_{\ell} D_{i\ell} r_{\ell}$, which is not identically zero because $D_{ij} \neq 0$. Fix all coordinates of r except r_j ; then L is a degree-one nonzero polynomial in the single variable r_j over the field $\mathbb{F}_p$, so it has at most one root. Hence $\Pr[L(r) = 0] \leq 1/p$, and $\Pr[Dr = 0] \leq \Pr[(Dr)_i = 0] \leq 1/p$. (This is the Schwartz–Zippel lemma [2, 3] specialized to degree one.) $\square$

Corollary 1 (Two challenges). *With k independent challenge vectors, a fabricated C passes all of them with probability at most $(1/p)^k$. For $p = 2^{61} - 1$ and $k = 2$,*

$$\left(\frac{1}{p}\right)^2 < 2^{-122}.$$

The Go verifier states and enforces exactly this bound (`crypto/poi/freivalds.go`): “Soundness error is $1/p \sim 2^{-61}$ per challenge vector; $k = 2$ vectors give $\leq 2^{-122}$.”

Theorem 2 (Post-quantum by construction). *The soundness of Theorem 1 and Corollary 1 is information-theoretic: it is a counting argument over a finite field and invokes no computational hardness assumption. A computationally unbounded adversary — in particular any quantum adversary — still fails to pass a beacon-derived challenge except with probability $\leq 2^{-122}$.*

This is the structural reason PoAI is post-quantum where zero-knowledge SNARK approaches are not: a SNARK’s soundness rests on, e.g., elliptic-curve or pairing assumptions that Shor’s algorithm breaks, whereas Freivalds’ soundness is the probability that a random low-degree polynomial vanishes, which no amount of computation reduces.

Theorem 3 (Zero false-reject). *If $C = A \cdot B$ exactly over $\mathbb{Z}$ and every entry of A, B, C is reduced into $[0, p)$ consistently, then (1) holds for every r . Thus an honest prover is never rejected.*

Proof. Field reduction is a ring homomorphism $\mathbb{Z} \rightarrow \mathbb{F}_p$; if $C = A \cdot B$ over $\mathbb{Z}$ then the identity holds after reduction, so $Dr = 0$ for all r . By Property 1 the honest integer accumulator is reproducible, so the operands the verifier reduces are exactly the prover’s. $\square$

Theorem 3 is what makes the scheme safe to put on the critical path of an economic system: an honest miner cannot be slashed by bad luck, and a challenger who submits a “fraud proof” against honest work simply finds the check passes.

2.4 Challenge unpredictability

A prover must not be able to pre-fit (A, B, C) to a known r . The canonical derivation hashes a public, post-commitment beacon together with the task and the committed transcript root; the verifier and prover derive the identical vectors (`poi.rs/freivalds.go, derive_challenges_keccak`):

$$r[j][i] = \text{BE_u64}(\text{KECCAK}(\text{seed} \parallel \text{BE32}(j) \parallel \text{BE64}(i))[0:8]) \bmod p,$$

with $\text{seed} = \text{KECCAK}(\text{beacon} \parallel \text{task} \parallel \text{root})$. Because the beacon is fixed only *after* the prover commits the root, the challenge is unpredictable at commit time; because the derivation is a pure function of public inputs, both sides reproduce it without interaction.

2.5 What the core proves, and what it does not

The Freivalds core proves a *single* matmul output is genuine. Two gaps remain, each closed by a later layer. First, a prover could commit *correct* matmuls on *unrelated* inputs — a valid-looking transcript that is not a real forward pass on the actual prompt; §4 closes this with graph chaining. Second, a prover could compute the matmul correctly but *return* a different value than it committed; §3.3 closes this by making the returned value a public function of the committed accumulator. The core is deliberately backend-agnostic and dependency-free so that its soundness can be unit-tested in isolation.

2.6 Verified test evidence for the core

The Rust core (`poi.rs`) and the Go core (`crypto/poi`) each ship a soundness suite that we ran to completion:

Test	Asserts	Lang
<code>honest_matmul_passes</code>	$C = A \cdot B$ passes every challenge	Rust/Go
<code>tampered_output_is_caught</code>	one flipped entry of C is rejected	Rust/Go
<code>negative_entries_field_reduction</code>	signed int8 reduces exactly	Rust
<code>larger_random_matmul_catches_sparse_cheat</code>	one off-by-one in 16×16 caught	Rust/Go
<code>challenge_derivation_deterministic</code>	both sides derive identical r	Rust
<code>wrong_dims_rejected</code>	shape mismatch fails closed	Rust/Go
<code>derive_challenges_keccak_golden_parity</code>	Rust $r =$ Go r , byte-for-byte	Rust/Go

Table 1: Representative soundness-core tests (all passing).

Running `go test ./...` in `crypto/poi` yields **22 passed, 0 failed**; running `cargo test -lib poi` in `hanzo-engine` yields **27 passed, 0 failed**. The Go suite also measures the verifier on the host machine at **1.13 ms** per opening verification, i.e. **882 verifications/second/node** single-threaded (`crypto/poi/scale_test.go, TestScale_VerificationThroughput`); this is a measured figure from that run, not a projection.

3 The Transcript: Committing a Forward Pass

A single matmul check is useless unless it is bound to a specific, committed computation. The transcript layer (`poi_transcript.rs`; Go mirror `crypto/poi/transcript.go`) makes “no valid compute proof, no mint” enforceable.

3.1 Commitment

Each proof-bearing matmul is committed as a leaf binding its three exact-integer operands under a domain tag:

$$\text{leaf} = \text{KECCAK}(\text{"hanzo/poi/matmul-leaf/v1"} \parallel \text{mat}(A) \parallel \text{mat}(B) \parallel \text{mat}(C)),$$

where $\text{mat}(M) = \text{BE32}(\text{rows}) \parallel \text{BE32}(\text{cols}) \parallel (\text{each } M_{ij} \text{ as BE64})$ is the canonical serialization. The leaves are folded into a binary KECCAK-Merkle root using the index-fold convention (even index $\rightarrow \text{KECCAK}(\text{node} \parallel \text{sibling})$, odd $\rightarrow \text{KECCAK}(\text{sibling} \parallel \text{node})$); odd levels duplicate the last node. This is the *transcript root* the prover posts on-chain.

3.2 Challenge and opening

A challenger derives the index of the matmul to open from the beacon and root,

$$\text{index} = \text{KECCAK}(\text{beacon} \parallel \text{root})[0:8] \bmod \text{length},$$

unpredictable before the prover commits the root. The prover *opens* that index, revealing (A, B, C) and the Merkle inclusion path. Verification is the conjunction of binding and correctness (`verify_opening`):

1. **Binding (Merkle inclusion).** The revealed operands hash to the committed leaf at `index` under `root`. A prover who swaps in different operands fails here.
2. **Correctness (Freivalds).** $C = A \cdot B$ under k beacon-bound challenge vectors derived from $\text{KECCAK}(\text{beacon} \parallel \text{root} \parallel \text{BE64}(\text{index}))$. A fabricated output fails here.

Lemma 1 (Opening soundness). *If `verify_opening` returns false for an opening that is Merkle-included under `root`, then with probability $\geq 1 - 2^{-122}$ the committed matmul at that index is fabricated ($C \neq A \cdot B$). Conversely an honest, included opening always returns true.*

This lemma is the production interface: the off-chain watcher imports `VerifyOpening` and slashes a bond exactly when it returns false on a Merkle-included opening, and the symmetric on-chain predicate `provesFraud` (§5.2) returns *true* on precisely that event.

3.3 Sound forward pass: the returned value is the committed value

The transcript binds operands, but an honest-looking transcript is worthless if the layer *returns* something other than what it committed. `ProvableLinear` (`poi_forward.rs`) closes this. It computes $y = \text{dequant}(\text{quant}(x) \cdot W_{i8})$ on the exact-integer accumulator, commits the operands ($A = x_{i8}$, $B = W_{i8}$, $C = x_{i8} \cdot W_{i8}$), and returns the dequantization of the *committed* C :

$$y[t][o] = C[t][o] \cdot \text{xscale}[t] \cdot \text{wscale}[o],$$

where the scales are public model parameters. Dequantization is therefore a public affine map of C .

Theorem 4 (Output-commitment binding). *Committing C commits y : anyone holding the committed C and the public scales recomputes y exactly, so a prover cannot claim an output that its committed matmuls did not produce.*

The test `test_output_is_the_committed_value` asserts the returned y equals $\text{dequant}(\text{committed } C)$ to within floating-point dequant tolerance, and `test_provable_ffn_emits_verifiable_transcript` runs a real SwiGLU feed-forward block (gate, up, down projections), confirms it emits exactly three committed matmuls, and verifies every one of them under a beacon-selected challenge.

4 Whole-Graph Composition: One Proof for Inference, Embedding, and Training

4.1 The gap the graph closes

Per-matmul soundness still permits a prover to commit individually-correct matmuls computed on activations it never derived from the prompt — splicing an unrelated tensor mid-stream. The graph layer (`poi_graph.rs`) makes the *wiring* of the computation part of the commitment.

Definition 1 (Computation trace). *A **GraphTrace** is a sequence of typed operations over hash-identified activations. An activation tensor M is named by its commitment $\text{act}(M) = \text{KECCAK}(\text{"hanzo/poi/act"} + \text{hash}(M))$. Each operation commits a leaf binding its kind, the commitments of its input activations, any inline parameters, and the commitment of its output activation. The trace also declares a set of input activations (the prompt, the weights, an upstream gradient).*

The supported operation kinds are exactly those a transformer and its backward pass require, each a pure deterministic integer function of its inputs:

Op	Semantics (exact integer)
MatMul	$C = A \cdot B$ — the only Freivalds-checked op
Add	residual out = $x + y$
ReLU	out = $\max(0, x)$
Scale	out = $(x \cdot \text{num})/\text{den}$ (e.g. attention’s $1/\sqrt{d}$)
TopKRoute	MoE router: top- k indices per row, lowest-index tie-break
Gather	expert dispatch: out[i] = $x[\text{idx}[i]]$
Softmax	deterministic fixed-point (Q16) integer softmax
Transpose	out[j][i] = $x[i][j]$ — the one op a backward pass adds

The softmax is the one subtlety: a floating-point softmax is not reproducible across backends, so the trace uses a pure integer scheme (max-shift for stability, an I-BERT-style fixed-point exponential [4], and an integer normalize) whose output is bit-identical for prover and challenger. The test `test_int_softmax_is_a_valid_distribution` confirms each row sums to $\approx 2^{16}$, is monotone in the logit, and is bit-reproducible.

4.2 The two checks

Verification of a trace composes two checks:

1. **Well-formed (chaining)**. Every operation’s input commitments reference either a declared trace input or the output commitment of an *earlier* operation. A break — an op reading an activation no prior op produced — is visible from the committed leaves alone, with no openings. `verify_chaining` returns false on the first dangling reference.
2. **Locally correct (per-op)**. An opened operation recomputes its output from its inputs — a MatMul by Freivalds under a beacon-bound challenge, a glue op (Add, ReLU, Scale, TopKRoute, Gather, Softmax, Transpose) by direct recomputation — and the recomputed result’s commitment equals the committed output.

4.3 Global soundness

Theorem 5 (Whole-graph soundness). *If a trace is well-formed and every operation is locally correct, then the committed final output equals the deterministic forward pass applied to the committed input.*

Proof. Induction over the topological order of the trace (the operations are recorded in execution order, which is a valid topological order). *Base:* a declared input activation’s committed value is, by definition, the true input. *Step:* consider operation o with inputs $u_1, \dots, u_m$. By chaining, each u_ℓ is either a declared input or the output of an earlier operation; by the inductive hypothesis its committed value is the true intermediate value. Local correctness of o guarantees that the committed output of o equals the operation applied to those true inputs. Since each op is the exact-integer deterministic function the reference forward pass uses, the committed output of o is the true intermediate value at o . Applying the step to the final operation yields the claim. $\square$

Because everything is exact integer, the activation commitments are reproducible and the glue recomputation is exact; in particular attention’s $QK^\top/AV$ matmuls and a complete MoE block (router $\rightarrow$ top- k $\rightarrow$ gather $\rightarrow$ expert matmuls $\rightarrow$ combine) are expressible with no floating-point reduction in the committed trace. The test `test_moe_graph_emits_and_every_op_verifies` traces a real MoE feed-forward block and verifies every op; `test_attention_scores_av_chain_verifies` does the same for an attention block including the integer softmax.

4.4 Training is a graph, not a separate proof system

The single operation a backward pass needs that a forward pass does not is **Transpose**: gradients are $dA = dC \cdot B^\top$ and $dB = A^\top \cdot dC$. With it, an SGD step is just a longer graph over the same ops — forward $Y = X \cdot W$; backward $dW = X^\top \cdot dY$, $dX = dY \cdot W^\top$; update $W' = W - \text{lr} \cdot dW$ via **Scale** and **Add**. The same **MatMul** op kind and the same Freivalds check cover both the forward (activation $\times$ parameter) and the backward (activation $\times$ activation) products; the trace exposes a `matmul_act` for the two-activation case.

Corollary 2 (One proof system). *Inference, embedding, and training are all instances of Theorem 5; only the shape of the graph differs, not the proof. A fabricated gradient is caught exactly as a fabricated inference output is.*

This is verified directly: `test_training_step_is_just_a_graph` traces a full linear-layer SGD step (seven ops: forward, two transposes, two backward matmuls, the scaled negation, the residual update), checks the graph chains, verifies every op, and then fabricates the weight gradient and confirms Freivalds catches it. Embedding is likewise a forward graph. Data-sharing is orthogonal to all of this: the graph’s committed *input* is its data provenance, so a proof of computation *is* a proof over the declared data without any separate mechanism.

4.5 Adversarial test evidence

The graph module ships a battery of attack tests, each asserting a specific cheat is caught:

Attack test	What it proves is caught
<code>attack_chain_splice_is_caught</code>	op reading an unproduced activation (chain)
<code>attack_tamper_output_at_open_is_caught</code>	claiming a different output at open time
<code>attack_fabricated_matmul_in_graph_caught_by_freivalds</code>	committing a faked matmul output
<code>attack_forged_softmax_is_caught</code>	tampering attention weights
<code>attack_forged_route_is_caught</code>	routing a token to a different expert

Table 2: Graph adversarial tests (all passing). A dedicated integration binary, `tests/poi_adversarial.rs`, adds further end-to-end attacks.

Across the four `poi` source modules, `cargo test -lib poi` runs **27 tests, 0 failed** (Property 1 through Corollary 2 are exercised by name in this set).

5 Tri-Language Byte Parity

A proof is only useful at settlement if the prover, the off-chain watcher, and the chain agree on every byte. PoAI is mirrored three ways and the mirrors are tested against each other, not merely asserted to match.

5.1 One hash, one field, one serialization

All three implementations use Ethereum KECCAK-256 (the same primitive as `luxfi/crypto.Keccak256` and Solidity `keccak256`), the same field $p = 2^{61} - 1$, the same domain tags, the same canonical (rows, cols, data) serialization, the same Merkle index-fold, and the same $\text{BE32}(j) \parallel \text{BE64}(i)$ challenge derivation. The Solidity verifier is pure (no external calls), so the same verification logic runs on any EVM.

5.2 The on-chain witness

`ComputeWitnessLib.sol` is the on-chain proof-of-inference witness check. Its core is `Freivalds` in pure Solidity, using `mulmod`/`addmod` under p so the field arithmetic is exact:

Listing 2: `Freivalds` over $\mathbb{F}_p$ on-chain (`ComputeWitnessLib.sol`).

```
function freivalds(Matrix memory a, Matrix memory b, Matrix memory c,
    uint256[] memory r)
    internal pure returns (bool)
{
    if (a.cols != b.rows || b.cols != c.cols || a.rows != c.rows || r.
        length != b.cols) return false;
    uint256[] memory br = _matvec(b, r); // length k
    uint256[] memory abr = _matvec(a, br); // length t
    uint256[] memory cr = _matvec(c, r); // length t
    for (uint256 i = 0; i < abr.length; i++) if (abr[i] != cr[i])
        return false;
    return true;
}
```

The library exposes two complementary predicates. `verifyOpening` returns true iff the opening is Merkle-included *and* `Freivalds`-valid (an honest `matmul`). `provesFraud` returns true iff the opening is Merkle-included *and* `Freivalds`-invalid (a committed-but-fabricated `matmul`) — this is the on-chain fraud proof: a challenger exhibits one such `matmul` from a prover’s committed pass, an honest pass has none, so an honest prover can never be slashed (the on-chain restatement of Theorem 3). The opened dimensions are hard-capped at `MAX_DIM = 1024` so a griefing prover cannot submit an unboundedly large opening; a full layer is challenged in bounded slices, and the witness stays small even though the model is large — the gas-side expression of `Freivalds`’ asymmetry.

5.3 Proven parity

Parity is enforced by golden tests, not by inspection. On the engine side, `test_derive_challenges_keccak_golden` pins the exact hex of the challenge vectors that the Go path emits and asserts the Rust path reproduces it byte-for-byte; the Go side asserts the same pinned value in `TestDeriveChallengesGolden`. On the contract side, the Foundry suite `ComputeWitness.t.sol` carries:

- `test_LeafParityWithProver` — the Solidity `matmulLeaf` equals the prover’s leaf;
- `test_GoOpeningVerifiesOnChain` — an opening produced by the Go verifier verifies inside the EVM unchanged;

- `test_FabricatedOutputCaughtOnChain` and `test_SwappedRevealCaughtOnChain` — the two attack classes are caught on-chain;
- `test_FreivaldsOverField` — the field arithmetic matches.

All five pass. This is the load-bearing interoperability claim: a proof emitted off-chain in Rust is verified on-chain in Solidity with no re-hashing and no re-encoding.

6 The AIVM as the Lux Settlement Layer

PoAI proves a computation; the AIVM turns a proof into settled value across the whole Lux network. The Lux flavor of the AIVM is precisely this: the virtual machine that *proves AI computation graphs* and *reconciles minting across all Lux chains*. Four contracts realize it, each owning one concern.

6.1 Mining a proof into a coin

`AIMiner.sol` turns one verified A-Chain inference receipt into a fair-launch subsidy mint, in pure Solidity so the same code runs on any EVM. The A-Chain (the AIVM’s attestation chain) settles an inference under its own consensus and provider quorum and commits a KECCAK-Merkle root over the resulting receipts. Mining checks, in order:

1. **Shape and finality.** The receipt is the canonical 355-byte encoding, version 1, status `COMPLETED`, with a non-zero output.
2. **Attestation.** The receipt’s leaf is recomputed byte-for-byte against the shared $A \leftrightarrow C$ wire spec and proven included under a root the chain has accepted.
3. **Compute proof (the load-bearing gate).** A `ComputeProof` must bind the receipt’s *own* measured model, prompt, output, and task — fields the submitter cannot forge because they are inside the Merkle-proven receipt — to a governance-accepted runtime. The contract *fails closed*: if no verifier is wired, no mint occurs. A forged root over a fabricated receipt is therefore insufficient; the compute proof certifying real (model, prompt, output) computation is mandatory.
4. **Global reconciliation.** The computation is claimed on the cross-chain ledger (§6.2).
5. **Mint.** Following checks-effects-interactions, the intent is marked consumed, the reward is clamped to the vested halving allowance, and the subsidy is minted to the receipt’s *bound requester* — never the submitter — so a relayer cannot redirect rewards.

The compute-proof binding itself lives in `ComputeProofLib.sol` as a two-step KECCAK chain that collapses (task, intent, model, prompt, open-block, operator, output, runtime) into one 32-byte `reportData` that every proof backend must reproduce. The pre-execution *challenge* fixes what was asked, to whom, and in what chain context *before* the model runs; the post-execution *report* attests what came out under which measured model and runtime. Folding the challenge into the report means a proof for task *A*/operator *X* can never be replayed for task *B*/operator *Y*. The `ComputeProof` struct carries a `proofType` (1 = confidential-compute TEE, 2 = zkML, 3 = optimistic Freivalds re-execution, 4 = *M*-of-*N* TEE), so the proof backend can be swapped without touching the economics; the binding (`reportData`) is identical across all four.

6.2 The global no-double-mint ledger

A proof’s on-chain binding includes the chain id, so a single proof cannot be replayed on another chain. But a prover could do the work *once* and submit distinct, chain-bound proofs of the *same* computation to many chains, minting the fair-launch subsidy N times for one piece of work. `AIComputeRegistry.sol` closes this. It keys on the *chain-independent* computation commitment

```
computationKey = KECCAK("hanzo/poi/compute-claim/v1" || modelSpec || promptHash || outputHash)
```

and records the first claimant. A second claim for the same (model, prompt, output) — from any miner, any chain — reverts. First-correct-proof wins; redundant recomputation earns nothing, which is exactly the Bitcoin-style incentive to do *novel* work rather than re-mine what is already settled. The miner claims inside its mint flow before the external mint (checks-effects-interactions), so a mint revert rolls back the claim. A claimer is always a proof-enforcing *contract*, never an externally-owned account — the same god-key defense as the coin’s mint seam.

This is verified by `test_FirstClaimWins_DoubleMintPreventedAcrossChains`, `test_ComputationKey_IsC`, `test_DistinctComputationsClaimIndependently`, `test_OnlyAuthorizedMinerCanClaim`, and `test_MinerMustBeContract` (all passing).

6.3 Trustless, permissionless, post-quantum cross-chain state

For the global ledger to be visible on every chain, each chain must learn the A-Chain’s committed roots without trusting a bridge. `AChainRootOracle.sol` provides this. Each epoch the A-Chain commits its receipt-root and consumed-set-root, and a *quorum* of A-Chain validators signs (root, height) with ML-DSA — the same post-quantum scheme `luxfi/warp` carries as an `MLDSACertSet`. *Anyone* may relay a signed root (permissionless, a public good), and the oracle verifies the quorum on-chain through the ML-DSA precompile: a relayer can only assert what the validators signed, never forge it, never mint. Acceptance requires that every key is a registered validator, the signer key hashes are strictly ascending (so no validator is double-counted), at least `threshold` signatures verify, and the height advances monotonically (no rollback).

Property 2 (Nation-state resistance). *Forging an accepted root requires either breaking ML-DSA (post-quantum hard) or corrupting at least `threshold` of the A-Chain validator set. This replaces the trusted single-relayer model with a verified validator quorum.*

This is verified by the `AChainRootOracle.t.sol` suite (7 tests, passing), which exercises quorum acceptance, sub-threshold rejection, non-ascending-key rejection, stale-height rejection, and unregistered-validator rejection.

6.4 Bonded providers and automated slashing

`MinerStakeRegistry.sol` requires a miner to bond capital in proportion to the compute capacity it declares; the bond is skin in the game. Three orthogonal authorities, one per concern:

- the **miner** controls its own bond (bond / request-unbond / withdraw), with a cooldown so a discrepancy can be proven before it exits;
- the **slasher** — the compute-proof fraud verifier — slashes a bond on a *proven* discrepancy, with no vote, just the math (the on-chain consumer of `provesFraud`);
- **governance** (a Safe) may blacklist a miner for off-protocol harm the math cannot catch.

A miner is **eligible** to mine iff bonded above a floor, bonded in proportion to declared capacity, not exiting, and not blacklisted. Slashing works *during* the unbond cooldown, so a miner cannot dodge a fraud proof by exiting first.

Property 3 (Operator-safe). *By Theorem 3, an honest miner’s openings always verify, so `provesFraud` can never return true against honest work; the slasher therefore cannot slash an honest bond. Slashing is reachable only by a Merkle-included opening whose Freivalds check fails — a genuinely fabricated matmul.*

The `MinerStakeRegistry.t.sol` suite (8 tests, passing) covers bonding, the capacity-proportional floor, the unbond cooldown, slasher-only slashing, and the blacklist veto.

7 Governance: A Leaderless Quorum of Operator LLMs

The AIVM is governed not by a council of humans but by the network’s own node operators reasoning with their language models, settling canonical decisions on-chain. The design is de-complexed across contracts, each one concern.

7.1 Sortition: leaderless committee selection

`Sortition.sol` selects a committee by cryptographic sortition. An operator is in the committee for a round iff its seeded ticket falls below a threshold sized so that, in expectation, $\sim \text{size of population}$ registered operators are selected:

$$\text{ticket} = \text{KECCAK}(\text{seed} \parallel \text{operator}), \quad \text{selected} \iff \text{ticket} < \left\lfloor \frac{2^{256}}{\text{population}} \right\rfloor \cdot \text{size}.$$

Selection probability is $\text{size}/\text{population}$, so an adversary controlling fraction f of the population controls $\sim f$ of the committee. To control a *majority* of a committee it must control a majority of the *whole registered population*, not merely race to fill the first `size` slots. This is the equal-vote regime — one operator, one ticket, no stake weighting, so a sub-threshold whale cannot buy disproportionate seats. The seed must be fixed *after* operators commit and be unpredictable to them (a future block hash or VRF beacon); membership is one KECCAK per check, so no on-chain enumeration of the operator set is needed.

Property 4 (Leaderless). *There is no sequencer, no privileged operator, and no leader. Committee membership is determined by post-commitment public randomness; the structural Sybil bar is a population majority, and the economic bar is a non-refundable registration cost.*

`Sortition.t.sol` verifies this with `test_ExpectedCommitteeSize`, `test_Deterministic`, and `test_CaptureRequiresPopulationMajority` (all passing).

7.2 Categorical decisions: AIGovernor

`AIGovernor.sol` produces a canonical YES/NO decision (with a confidence bucket) by aggregating signed, structured operator-LLM verdicts. Each verdict is authenticated by `ecrecover` over the exact canonical preimage, the recovered signer must equal the sender, signatures are low- S with $v \in \{27, 28\}$, and one verdict is admitted per operator per task. To defeat copying, a commit-reveal mode seals each operator’s commit during a commit window and admits the reveal *strictly after* the window closes, so no operator can observe a peer’s revealed fields before its own commit is sealed; the commit is operator-bound. `settle` has no access modifier — it is permissionless, gated only by the state machine and a deadline (so no one can front-run the threshold-th honest vote to suppress a forming quorum). The tally selects the largest group by consensus key; if it meets the threshold (a strict majority, $\geq n/2 + 1$), the decision settles. There is no leader anywhere in this path.

7.3 Continuous values: AIParams

Where the governor decides a categorical policy, `AIParams.sol` decides a continuous *value*: each operator’s LLM proposes a number within a declared range, and the chain settles to the *median* of a quorum of proposals and makes it the live knob value. The median is deliberately unweighted — one operator, one proposal — and is Byzantine-robust to any minority below 50%. The committee is sortition-selected with a post-commitment seed (the open-block hash), the population is snapshotted at open, and `settle` is again permissionless. `AIParams.t.sol` (16 tests, passing) verifies the median computation, the in-range invariant, and the sortition gate.

7.4 From a settled decision to an arbitrary on-chain action

`AIExecute.sol` is the single surface validator consensus uses to read and act on-chain, decomplexed by risk into three tiers, one mechanism:

- **Read.** Any getter is `staticcall`’d and handed back as a typed value (a bool, a number, an address, a word), so consensus can query arbitrary on-chain state structured.
- **Enact (Tier 1, no timelock).** A knob the validators have *decided* in `AIParams` is spliced as the sole argument of `target.selector(value)`; a 32-byte word ABI-encodes to any single type, so one call sets a bool / uint / int / address / bytes32. Low-risk and idempotent because the value *is* the consensus output.
- **Execute (Tier 2/3, windowed).** An *arbitrary* operation — any target, any method, any arguments, any native value — that the validators approved by its hash, gated by a timelock window, predecessor ordering, one-shot replay protection, a guardian veto, and an optional policy guard.

The operation hash binds `chainid` and the executor instance, so an approval can never be replayed on another chain or another executor.

The authority for execution is supplied by `AIApproval.sol`, an adapter (not a second governor) that binds a settled YES verdict to an operation through a single identity:

a Thought’s `promptHash` $\equiv$ the `AIExecute` operation hash.

So “should consensus execute operation *X*?” is literally the governance question the operators answered. An opener poses a Thought whose `promptHash` equals `hashOperation(op)`; the operators run their LLMs and settle; if the decision is `SETTLED` with vote YES, anyone may permissionlessly `confirm` it, which stamps the approval at the current block time — and *that* timestamp is where the timelock window begins. There is no admin, no owner, and no override: a NO, abstain, unsettled, or non-existent Thought yields no approval, so `execute` reverts. The only path to an approval is a real quorum YES. `AIApproval.t.sol` (7 tests, passing) verifies the settled-YES binding, the one-shot guard, and the rejection of every non-YES state.

7.5 Governance as a second proof of useful cognition

Consensus itself is useful cognition, and the network mints for it too — but only when it is backed by proven compute. A vote is ~ 1 bit, so “guess the modal YES/NO” must not mint. `AIConsensusMiner.sol` closes this: a winning validator is paid only if it has *also* submitted a valid `ComputeProof` binding the settled Thought’s (task, model, prompt) to that validator’s own output. Consensus says “they agreed”; the compute proof says “they computed”; both, or no mint. The reward is split among the proven winners and goes to them, never to the caller. Critically, `AICoin` shares one `mintedSubsidy` counter across both mint paths (the attestation miner of §6.1 and this governance miner), so the halving schedule bounds them *jointly* — adding a second proof-of-cognition mint seam never inflates past the cap (§8). `AIGovernanceBridge.sol`

additionally turns each settled decision into a permanent, indexable Proof-of-Thought receipt in `AIThoughtRegistry.sol`, so the cognitive ledger is queryable.

8 Economics: Bitcoin-Shaped, Fair-Mined on Useful Work

8.1 The issuance schedule

`AICoin.sol` is the pure value layer; it knows nothing of tasks, receipts, or governance — only the issuance schedule and a single `mint` seam. Its parameters are Bitcoin-shaped:

- a hard cap `MAX_SUBSIDY` = 1,000,000,000 coins, with *no pre-mine*: supply starts at zero and every coin is minted to a cognitive node for accepted work;
- a halving period `HALVING_PERIOD` = 4×365 days, so the emission slope halves every four years.

Within an epoch the allowance vests linearly and the slope halves each epoch. With $k = \lfloor (t - \text{genesis}) / \text{HALVING_PERIOD} \rfloor$ the current epoch and $f \in [0, 1)$ the fractional progress through it, the cumulative allowance is

$$\text{allowed}(t) = \text{MAX} (1 - 2^{-k}) + f \cdot \text{MAX} 2^{-(k+1)}. \quad (3)$$

Theorem 6 (Capped, continuous, exact). *allowed(genesis) = 0; allowed is continuous and strictly increasing; and allowed(t) → MAX as t → ∞. The geometric sum of per-epoch emissions equals exactly MAX: $\sum_{k \geq 0} \text{MAX} \cdot 2^{-(k+1)} = \text{MAX}$.*

The mint function clamps every mint to $\text{allowed}(t) - \text{mintedSubsidy}$, so cumulative minted can never exceed $(3) \leq \text{MAX}$ — the cap is structurally enforced, not merely checked. The contract handles a unified network-wide genesis that may sit *ahead* of a given chain’s clock by returning a zero allowance before genesis rather than underflowing, so the same coin emits on one shared curve across every EVM that mints it.

8.2 Deflation

`AICoin` is burnable, and the protocol burns a governed fraction of every settled fee. Net supply change per block is $\text{subsidy}(t) - \text{burn}(t)$; since $\text{subsidy} \rightarrow 0$ while fee burn persists under use, net supply turns deflationary once the chain is in steady use.

8.3 Why this is the next Bitcoin

Bitcoin’s social contract is a capped, fairly-distributed asset whose issuance is earned by verifiable work that no one can counterfeit. PoAI keeps the contract and replaces the substrate:

- **Capped and fair-mined** (Theorem 6, `test_NoPremine`, `test_FairMineEntireCapThenFeesOnly`): one billion, no pre-mine, halving, deflationary tail.
- **Public and permissionless** (§6.1, 6.2): anyone may submit a valid (receipt, proof) pair and the subsidy mints to the bound requester; first-correct-proof wins.
- **Leaderless** (Property 4): no sequencer, no privileged operator; governance is sortition over the whole operator population.
- **Operator-safe** (Property 3): bond-and-slash, never exploitable against honest work, by the zero-false-reject theorem.
- **Post-quantum / nation-state-proof** (Theorem 2, Property 2): the proof’s soundness is information-theoretic, and cross-chain trust rests on a post-quantum validator quorum.

The decisive difference is that the work is *useful*. Bitcoin spends exahashes computing partial preimages whose only value is their difficulty. PoAI spends the same economic effort on inference, embedding, and training that the world wants performed anyway, and proves — information-theoretically, cheaply, and on-chain — that the work was done correctly. The hashing is not wasted; it *is* the AI.

9 Verified Test Evidence (Aggregate)

Every property in this paper is grounded in shipped, tested code that we executed to completion. No benchmark or count below is projected.

Suite	Result	Command
crypto/poi (Go verifier)	22 passed, 0 failed	<code>go test ./...</code>
hanzo-engine poi (Rust core)	27 passed, 0 failed	<code>cargo test -lib poi</code>
contracts/ai (Solidity, 20 suites)	256 passed, 0 failed	<code>forge test</code>

Table 3: Test results, all green. The Solidity figure spans AICoin, AIMiner, AIComputeRegistry, AChainRootOracle, MinerStakeRegistry, ComputeWitness, ComputeProofEnforcement, AIApproval, Sortition, AIGovernor (and its commit-reveal and red-team suites), AIParams, AIConsensusMiner, AIReputation, AIGovernanceBridge, AIObservatory, and the Governance suite.

The single measured performance figure is the Go verifier’s opening-verification latency: **1.13 ms**, i.e. **882 verifications/second** on one thread of the test host (`scale_test.go`). With N independent verifying nodes the aggregate scales linearly while the probability of catching a fraud committed at honest-fraction 0.1 rises toward 1 as challenges accumulate, exactly as the suite’s printed table reports.

10 Post-Quantum Primitives

The AIVM is post-quantum throughout, anchored in the standardized lattice and hash-based schemes, registered as EVM precompiles in the LP-4200 unified PQCrypto address block:

Primitive	Address	Role
ML-KEM (FIPS 203)	0x012201	module-LWE KEM
ML-DSA (FIPS 204)	0x012202	module-LWE signature; the AChainRootOracle verifier
SLH-DSA (FIPS 205)	0x012203	hash-based signature
Pulsar	0x012204	module-LWE threshold (FIPS 204 byte-equal)
P3Q	0x012205	post-quantum rollup-commit threshold verifier (LP-218)
X-Wing KEM	0x002221	X25519 + ML-KEM-768 hybrid (encapsulate)

Table 4: Post-quantum precompiles (verified addresses from each precompile’s `contract.go`).

The ML-DSA precompile (default mode ML-DSA-65, NIST Level 3) is the verifier `AChainRootOracle` invokes for the cross-chain root quorum (§6.3); the address is supplied to the oracle by deployment configuration rather than hard-coded in Solidity. The P3Q precompile at 0x012205 verifies a Pulsar-threshold signature for the LP-218 rollup commit and dispatches through the same FIPS-204 ML-DSA verifier core. `luxfi/warp` carries an optional `MLDSACertSet` on its envelope and refuses to downgrade in strict-PQ mode.

Prompt confidentiality for proof-of-inference is provided by `luxfi/crypto/promptseal` — a pure-Go HPKE envelope (not a precompile) that seals a user’s prompt to an operator’s registered

public key under the X-Wing hybrid KEM (X25519 + ML-KEM-768), HKDF-SHA256, and ChaCha20-Poly1305, with domain tag "hanzo/poi/prompt-seal/v1". Hybrid means the seal stays confidential if *either* X25519 *or* ML-KEM-768 holds; only the chosen operator, inside its compute boundary, can open it. The matching on-chain surface is the X-Wing *encapsulate* precompile at 0x002221; key generation and decapsulation are intentionally excluded on-chain because they would expose secret material, and run off-chain in the `promptseal` library.

11 Related Work

zkML. Zero-knowledge proofs of ML inference [6] give a succinct non-interactive proof but at $100\text{--}1000\times$ prover overhead, and their soundness rests on computational assumptions (pairings, discrete log) that a quantum adversary breaks. PoAI’s interactive Freivalds check is an order of magnitude *cheaper* than recomputation rather than orders of magnitude more expensive, and its soundness is information-theoretic (Theorem 2). PoAI also natively spans training (§4.4), which most zkML systems do not.

Trusted execution environments. TEE attestation (SGX, SEV-SNP, H100 confidential computing) assumes hardware trust and has a history of microarchitectural breaks. The AIVM *accommodates* TEE evidence as one `ComputeProof.proofType` (§6.1) but does not *depend* on it: the canonical, fail-closed path is the software-only, hardware-independent Freivalds witness.

Optimistic rollups. PoAI’s fraud-proof structure — commit a root, allow anyone to challenge an opening within a window, slash on a proven discrepancy — shares optimistic rollups’ philosophy [7], specialized to matrix computation where the fraud proof is a single Freivalds check with $O(\log n)$ Merkle overhead rather than a full re-execution.

Proof of work. Bitcoin [5] settles useless work; PoAI keeps the capped, fair-mined, leaderless economics and substitutes useful, cryptographically verified AI computation (§8).

12 Conclusion

Proof of AI is the settlement primitive a decentralized AI network needs: a cheap-to-verify, expensive-to-fake, information-theoretically sound proof that an AI computation graph ran correctly, where the work is the product. Because inference, embedding, and training are all graphs, it is one proof system rather than three; because it is anchored on the exact-integer accumulator of a quantized model, it is bit-exact, deterministic, and zero-false-reject; and because its soundness is a counting argument over a finite field, it is post-quantum by construction. The AIVM builds the full economic machine on top: a global no-double-mint ledger, a trustless post-quantum cross-chain relay, bonded providers with automated slashing, a leaderless sortition-quorum of operator LLMs that settles canonical decisions and executes arbitrary approved operations, and a Bitcoin-shaped, fair-mined coin whose subsidy is earned entirely by verified cognition. Every claim here is backed by shipped code and passing tests — 22 in Go, 27 in Rust, 256 in Solidity, all green — and one measured benchmark of 1.13 ms per on-chain equivalent verification. The hashing is not wasted. It is the AI, and it is proven.

Acknowledgments

This work was supported by Hanzo Industries, Lux Industries, and the Zoo Labs Foundation. The canonical contract implementation lives at `luxfi/standard` under `contracts/ai/`; the soundness core lives in `hanzo-engine` (`src/poi*.rs`) and `luxfi/crypto` (`crypto/poi/`).

References

- [1] R. Freivalds, *Probabilistic machines can use less running time*, Information Processing (IFIP Congress), 1977, pp. 839–842.
- [2] J. T. Schwartz, *Fast probabilistic algorithms for verification of polynomial identities*, Journal of the ACM 27(4), 1980, pp. 701–717.
- [3] R. Zippel, *Probabilistic algorithms for sparse polynomials*, in Symbolic and Algebraic Computation (EUROSAM), 1979, pp. 216–226.
- [4] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, K. Keutzer, *I-BERT: Integer-only BERT Quantization*, ICML, 2021.
- [5] S. Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System*, 2008.
- [6] D. Kang, T. Hashimoto, I. Stoica, Y. Sun, *Scaling up Trustless DNN Inference with Zero-Knowledge Proofs*, arXiv:2210.08674, 2022.
- [7] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, E. W. Felten, *Arbitrum: Scalable, private smart contracts*, USENIX Security, 2018.
- [8] National Institute of Standards and Technology, *FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA)*, 2024.
- [9] National Institute of Standards and Technology, *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)*, 2024.
- [10] M. Barbosa, D. Connolly, et al., *X-Wing: The Hybrid KEM You’ve Been Looking For*, IACR ePrint 2024/039, 2024.